

Nome: João Marcos Melo Monteiro

Blueprint de Engenharia: Pipeline Híbrido de Transpilação e Otimização Semântica (PL/pgSQL → Python 3.14)1. Topologia do Grafo e Gestão de Estado EstritoEm vez de tratar o fluxo como uma simples sequência de passos, o encaramos como um Transpilador Híbrido com Feedback Loop. O estado no LangGraph não deve apenas acumular dados soltos, mas funcionar como uma tabela de símbolos e uma representação intermediária (IR).Pythonfrom typing import TypedDict, Optional, Dict, Any, List

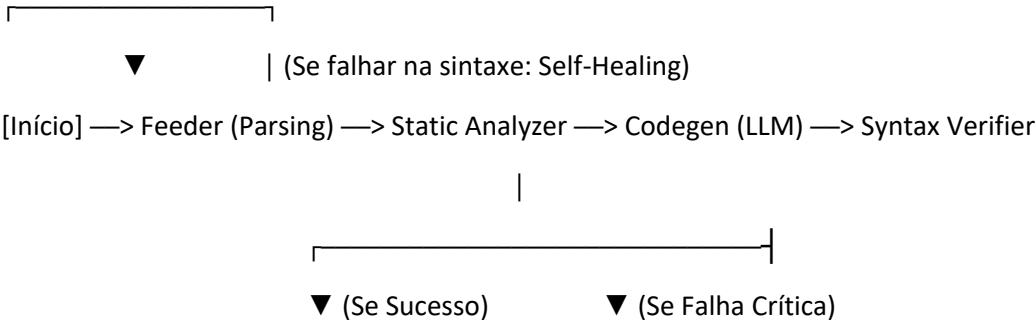
```
class TranspilerState(TypedDict):
    source_code: str          # Código-fonte original em PL/pgSQL
    schema_context: Optional[str] # DDL/Schema para ancoragem contextual
    target_version: str       # Default: "3.14"

    # Representação Intermediária (IR) e Metadados
    ir_ast: Dict[str, Any]    # Árvore de Sintaxe Abstrata gerada pelo sqlglot
    detected_antipatterns: List[str] # Lista de riscos estruturais identificados

    # Artefatos de Saída e Validação
    generated_python_code: Optional[str]
    compiler_diagnostics: Dict[str, Any] # Erros de sintaxe/Linter (ast.parse)
    evaluation_metrics: Dict[str, Any] # LLM-as-a-judge (Mapeamento Semântico)

    execution_status: str     # "PENDING", "PARSED", "GENERATED", "VALIDATED", "FAILED"
```

O Fluxo de Controle Dinâmico (Orquestração Baseada em Estado)Implementaremos uma Edge Conditional após a validação estática. Se o linter ou o analisador ast do Python capturar um erro sintático (gerado por alucinação da LLM), o grafo faz o rollback do estado e aciona um nó de correção (Self-Healing Node), retroalimentando a LLM com o log do erro de compilação.



Persist & Evaluate —> [Fim] Fail State —> [Fim]

✂ 2. Arquitetura dos Nós: Da Análise Sintática à Geração Eficiente
Nó 1: Front-end do Compilador (Parsing com sqlglot) Aqui, em vez de um bloco try/except genérico, fazemos uma varredura cirúrgica (Walking the AST). O sqlglot decompõe a query e extrai a tabela de símbolos (procedimentos, variáveis locais e tabelas mutadas).
`Pythonimport sqlglot`

```
from sqlglot import exp
```

```
def node_ast_parsing(state: TranspilerState) -> dict:
```

```
    try:
```

```
        # Parsing estrito configurado para dialeto Postgres
```

```
        expression = sqlglot.parse_one(state["source_code"], read="postgres")
```

```
        # Extração de metadados de forma determinística (Ex: tabelas que sofrem mutação)
```

```
        tables = [table.sql() for table in expression.find_all(exp.Table)]
```

```
    return {
```

```
        "ir_ast": { "success": True, "target_tables": list(set(tables)), "tree_root":  
repr(expression)},
```

```
        "execution_status": "PARSED"
```

```
    }
```

```
    except Exception as err:
```

```
        return {
```

```
            "ir_ast": { "success": False, "error": f"Lexer/Parser Error: {str(err)}" },
```

```
            "execution_status": "FAILED"
```

```
        }
```

Nó 2: Static Code Analysis (Mapeamento de Impedância e Antipadrões) Este nó funciona como as passes de otimização de um compilador. Antes de tocar na LLM, injetamos heurísticas puras no pipeline para interceptar armadilhas clássicas de conversão de banco de dados para memória: Detecção de Iteração Procedural (Cursors Explícitos): Identifica estruturas DECLARE ... CURSOR e loops FETCH. O relatório semântico deve forçar o prompt a converter isso em operações vetoriais/set-based (Pandas, polars ou SQLAlchemy Core bulk inserts), mitigando o gargalo de latência e chamadas repetitivas ao banco (\$N+1\$). Acoplamento de Estado Recursivo (CTEs): Mapeia cláusulas WITH RECURSIVE para garantir que o código Python gerado utilize recursão em cauda ou estruturas de pilha controladas para evitar estouro de memória (Stack Overflow). Tratamento de Exceções Relacionais: Identifica blocos EXCEPTION WHEN para induzir a geração de um gerenciador de contexto (contextmanager) robusto em Python que garanta o

rollback atômico da transação caso a aplicação caia no meio de um lote.

Nó 3: Codegen Engine (Prompt Engineering Baseado em Contexto Livre)O segredo do prompt aqui é a Ancoragem de Contexto. Passamos o DDL das tabelas envolvidas (Anexo A) e o relatório de antipadrões gerado no Nó 2. Instruímos a LLM a atuar como um engenheiro de software focado em alta performance.

Tese Arquitetural para a Banca Examinadora: > "A tradução ingênua de PL/pgSQL para Python frequentemente introduz gargalos catastróficos ao tentar emular processamento orientado a registros na memória da aplicação. Minha abordagem arquitetural prioriza a separação de escopos: a lógica de controle de fluxo e as regras de negócio residem na camada Python (garantindo testabilidade e extensibilidade), enquanto a manipulação pesada de dados permanece encapsulada em queries otimizadas enviadas em lote (Batch) ao SGBD via SQLAlchemy, preservando a eficiência do motor relacional."

Nó 4: Verificação Sintática e Grafo de Sintaxe do DestinoUsamos o módulo ast nativo do Python para compilar dinamicamente o código gerado em uma árvore de execução local. Se o parser do Python rejeitar o código, capturamos a linha e a coluna exata do erro sintático.

3. Camada de Infraestrutura e Persistência PoliglotaA tabela de histórico usa recursos nativos do Postgres para auditoria avançada. Ao tipar o campo de relatório como JSONB, conseguimos indexar via GIN para criar telemetria de quais antipadrões ocorrem com maior frequência nos legados analisados.

```
SQLCREATE TABLE modernization_history (
```

```
id BIGSERIAL PRIMARY KEY,
```

```
source_code TEXT NOT NULL,
```

```
generated_code TEXT,
```

```
diagnostics JSONB NOT NULL,
```

```
execution_status VARCHAR(30) NOT NULL,
```

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW())
```

```
);
```

-- Índice GIN para consultas rápidas na árvore JSONB de telemetria

```
CREATE INDEX idx_modernization_diagnostics ON modernization_history USING gin
(diagnostics);
```

Orquestração de Ambiente (Docker Compose)Desenhamos o ecossistema com isolamento de rede:db-service: Instância PostgreSQL isolada com volumes persistentes para simular o banco legado e o repositório da aplicação.observability-hub (Langfuse): Coleta e unifica os traces de execução de cada nó do LangGraph de forma transparente, permitindo auditoria de chamadas assíncronas e análise do custo financeiro de tokens.transpiler-api (FastAPI): Motor assíncrono que expõe endpoints de saúde (/health) e de conversão (/api/v1/modernize).

4. Diferencial de Engenharia (Acelerador de Bônus: Evaluation)Para cravar a nota máxima e demonstrar domínio completo sobre o ciclo de vida de IA gerativa, aplicamos o conceito de LLM-as-a-judge focado em Semântica Relacional (Bônus 3).A Métrica: Transactional Mismatch Index (TMI) e Syntax Rate.A Lógica: Após a aprovação do código pelo compilador (ast.parse), acionamos uma LLM em um escopo isolado de avaliação. Fornecemos o código original e o gerado sob um gabarito de avaliação rigoroso. O avaliador checa critérios objetivos:O comportamento de rollback foi mantido?As restrições de chaves primárias e integridade referencial estão

protegidas na lógica em Python? Houve injeção de loops desnecessários? Implementação no Trace: Esse score é injetado como um metadado numérico (score) na API do Langfuse ligado diretamente ao ID da execução. Durante a entrevista técnica, você terá um painel visual quantitativo provando a taxa de acerto sintático e semântico do seu pipeline. Planejamento de Execução Técnica: Fase Inicial (Bootstrap): Scaffold da arquitetura de pastas limpa (/app/core, /app/graph, /app/db, /tests). Fase de Testes de Sanidade (TDD): Implementação dos casos base dos Anexos B e C utilizando pytest para blindar o comportamento do parser local antes de realizar qualquer requisição externa de API de LLM.